

Substrate-Level Conflict Preservation as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 2 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the two levels of conflict handling named in the source paper's "conflict-preserving" commitment — substrate-level preservation — as a standalone architectural commitment with independent operational content, separable from the cell-level resolution mechanism with which it composes.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "conflict-preserving" commitment names a two-level handling pattern: contradictions persist at the substrate level by default, and cells executing over the substrate resolve them at execution time according to human-authored orchestration rules. A separate note formalizes that joint commitment. This note formalizes the first half of the pair — substrate-level preservation — as having independent architectural content that can be defended, implemented, and tested independently of cell behavior. The motivation is concrete: substrate-only audits, substrate-as-archive evaluations, substrate migration between hosts, and compliance assessments examine the substrate's state without reference to runtime cell execution, and each requires a precise specification of what substrate-level preservation requires of the substrate alone. The note states the five operational components of the commitment, identifies what the commitment does not require, distinguishes it from four adjacent technical patterns commonly conflated with it, names six downstream commitments the substrate level is load-bearing for, enumerates seven failure modes that violate it specifically, and provides an operational test for whether a given substrate satisfies it.

1. Why substrate-level preservation needs to be formalized as standalone

The CKS pattern's "conflict-preserving" commitment names a two-level pattern: substrate-level preservation as architectural default and cell-level resolution under orchestration rules at execution time. A separate derivation note formalizes that joint commitment as the architectural promotion of contradiction to first-class substrate object with two coupled levels of handling.

The joint framing is correct as far as it goes, and this note does not contradict it. But several deployment situations evaluate the substrate's preservation properties without reference to cell behavior at all. A compliance auditor examining a substrate-as-archive assesses what the substrate carries, not what cells executing over it would do. A substrate migration between hosts requires the receiving host to preserve

contradictions in the state it inherits, regardless of which cells run against the new host. A separation-of-duties deployment may delegate substrate hosting to one party and cell execution to another, with each party's commitment specifiable independently of the other's. If substrate-level preservation is treated only as half of a composite that requires cell-level execution to be meaningful, these cases have no principled architectural vocabulary.

A second motivation is that substrate-level preservation is the commitment most often violated by seemingly-reasonable simplifications. Implementations under pressure to provide "clean" data treat contradictions as data quality problems to be resolved at write time — through deduplication, merge-on-write, last-writer-wins, schema-validation rejection of contradicting writes, or replica reconciliation. Each is a reasonable engineering choice in some other architectural context (databases, distributed stores, version control, event sourcing); in CKS, each violates a load-bearing commitment, and the violation usually happens silently during ordinary operation rather than as a deliberate architectural choice. Naming the standalone commitment with precise operational content is what makes those violations identifiable as such.

2. The substrate-level preservation commitment, defined precisely

In the CKS pattern, a substrate satisfies **substrate-level conflict preservation** if and only if all of the following hold during the substrate's existence.

(a) Contradicting content exists as substrate content. When two pieces of substrate content contradict each other, both remain in the substrate as substrate content, with the substrate-state properties the source paper assigns to first-class content (persistence, addressability, authority, provenance). Neither contradicting piece is silently removed, marked as superseded outside the architectural authority structure, or hidden from readers exercising the inspect right over the underlying state. Both pieces are addressable by reference and readable in inspectable form.

(b) Contradictions persist until resolved by human authority or human-authored orchestration rule. A contradiction remains substrate content until either (i) a human with appropriate governance authority directly modifies the substrate to resolve it, or (ii) an orchestration rule that humans authored handles the contradiction at cell execution time (the cell-level resolution treated in the companion note A2.14). The resolution path is architecturally human-authored; no other path for contradiction collapse exists architecturally.

(c) Contradictions are not collapsed by LLM operations. When an LLM operating in the substrate-mediator role reads contradicting substrate content, it does not modify the underlying contradicting content as a side effect of reading it. The LLM may, under orchestration rules, produce an output recorded as substrate content that addresses the contradiction; what it cannot do is silently remove or transform the contradicting content already present.

(d) Contradictions are not collapsed by automated processes. Automated processes operating over the substrate — deduplication routines, merge workers, consistency reconcilers, garbage collectors, schema validators, compaction processes — do not remove or transform contradicting content. Such processes may operate over the substrate to perform technical functions consistent with the architectural commitments, but they have no authority to collapse contradictions; collapse requires

human authority or a human-authored orchestration rule, by (b).

(e) Contradictions are not collapsed by runtime middleware. Layers between humans and the substrate — access proxies, observability layers, governance gateways, replication middleware — do not transform substrate content as it is read or written in ways that remove contradictions. Contradictions visible in the substrate are visible to readers; writes that produce contradictions are committed as contradictions, not silently merged at the middleware layer.

The five components together define what substrate-level preservation requires architecturally. A substrate that satisfies fewer than five preserves contradictions partially, not in the architectural sense the commitment names.

A clarification on the scope of "conflict." The commitment is broader than contradicting facts. It covers contradicting decisions (two cells writing different decisions about the same coordination question), contradicting authorities (two humans with override authority making opposing changes), contradicting interpretations (the same substrate content read by different cells producing different downstream decisions and recorded as substrate content), and contradicting rules (two orchestration rules with overlapping scope giving incompatible guidance). The commitment is preservation-regardless-of-type. A deployment may add type metadata as substrate schema; the architectural commitment makes no distinction by type.

3. What the commitment does NOT require

The standalone treatment is not a maximalist treatment. Stating what the commitment does not require is what keeps the framing from drifting beyond what the source paper supports.

It does not require that contradictions be held forever. Resolution through human authority or human-authored orchestration rule is permitted; the commitment is about *how* resolution happens, not that resolution is forbidden. What the commitment forbids is silent collapse outside the architectural authority structure, not change under that structure.

It does not require contradictions to be visually prominent in user interfaces. The commitment is about substrate state, not about user experience over that state. Interfaces may render substrate content in any way the deployment chooses, including hiding contradictions in default views, provided the underlying state preserves them and humans exercising the inspect right can reach the underlying state directly.

It does not require automatic detection of contradictions. Detection is a deployment choice. An orchestration rule may flag potential contradictions at write time, or contradictions may surface only through later inspection. The commitment is to preservation once contradictions exist, not to active detection of contradictions as they arise.

It does not require contradictions to carry inherent type metadata. A substrate that preserves contradictions without architecturally distinguishing decisions versus interpretations versus rules satisfies the commitment. Type metadata is a deployment-level enrichment, not an architectural requirement.

It does not subsume the relationship-metadata requirement. Contradicting content carries provenance metadata, including a relationship field that names what each piece contradicts (the four-field provenance specification is treated separately in the companion note A2.16). The relationship metadata is a property contradicting content has; it is not a substitute for preserving the underlying content. A substrate that recorded only relationship metadata while collapsing the contradicting content itself would fail the commitment regardless of how rich the metadata.

4. What substrate-level preservation is NOT

Four adjacent technical patterns are commonly conflated with substrate-level preservation. Each is a real and reasonable commitment in some other architecture; naming what preservation is not is what prevents the misreading.

Not write-conflict handling in databases. Database write-conflict handling addresses what happens when two concurrent transactions modify the same record — typically resolved through locking, multi-version concurrency control, or last-writer-wins. The architectural commitment in CKS is different: it addresses what happens to substrate content that contradicts other substrate content, regardless of when each was written. Database write-conflict mechanisms may be used as the technical realization of substrate writes; what those mechanisms do at the technical level (lock-then-write, version reconciliation) does not satisfy the architectural commitment by itself, because the architectural commitment is about preserving contradictions in the substrate state that *results* from writes, not about how concurrent writes are coordinated technically.

Not version-control merge conflicts. Version control systems surface merge conflicts when concurrent edits to the same file overlap. Once resolved through human merge-conflict resolution, the conflict is gone from the working state — the resolved file replaces the conflicting versions, and the conflict survives only as historical state in the version graph. The architectural commitment in CKS is different: substrate content with contradictions preserves the contradiction in the *working* state, not only in history. A version-control-style "the merge has been resolved" model where the contradiction disappears from current substrate state is not CKS-coherent.

Not eventual consistency in distributed systems. Eventual-consistency models accept temporary inconsistency between replicas with the understanding that consistency will eventually be achieved through reconciliation. The architectural commitment in CKS is that contradictions are not transient inconsistencies to be reconciled away; they are first-class content to be preserved. A substrate operating under eventual consistency would still need to preserve contradictions as architectural state once consistency converged, not collapse them as part of the consistency mechanism.

Not audit logging of changes. Audit logs record changes to state over time — what was modified, by whom, when. The architectural commitment in CKS is not about logging changes; it is about the *current* state of the substrate carrying contradictions as authoritative content. A substrate with comprehensive audit logging that nonetheless silently resolves contradictions in its current state has logs of the resolution but no preservation in the architectural sense. The substrate is the source of truth for what conflicts remain unresolved; audit logs are the source of truth for what changed, which is a different category.

5. Why substrate-level preservation is load-bearing for downstream commitments

Substrate-level preservation is load-bearing for six other commitments in the CKS pattern. The commitment cannot be removed or weakened without consequences elsewhere.

Source of truth. The substrate is authoritative for "what conflicts remain unresolved" — one of the categories of authoritative state the source paper names (§11.3). Without preservation, this category disappears: the substrate cannot be authoritative about conflicts it has silently resolved.

Path retraceability. The retraceable path of a coordination decision must include the contradictions that contributed to it or that the decision selected against (§3.1). Without preservation, decision paths traversing contradictions would have invisible nodes; retraceability fails for any decision that involved contradiction handling.

Determinism. The substrate is committed to deterministic state behavior at the coordination layer (§4.1, §11.3). Silent collapse violates this: two reads of the same logical question yield different downstream results depending on which contradicting piece was silently chosen, and the choice is not itself substrate state. Preservation is what makes determinism observable for content that contradicts other content.

Cell-level resolution. Cells resolve contradictions under orchestration rules at execution time (§5.3); the architectural commitment is that the resolution is a cell-level decision recorded as new substrate content, not a substrate-level collapse of the underlying contradicting content. Preservation is the precondition: cell-level resolution decisions must be recordable as decisions *about* a contradiction that still exists in the substrate.

Two-level coupling. The two-level handling pattern requires both levels. If substrate preservation fails, cell-level resolution has nothing to operate on as preserved state — it becomes the only level, and the two-level pattern collapses into single-level resolution-at-write-time, which is exactly what the commitment was designed to prevent.

KO/OIDA inheritance. OIDA's signed contradiction edges are the cited prior art for treating contradictions as first-class objects with relationships (§6.2). Substrate-level preservation is what inherits this property architecturally; without preservation in the substrate's state, the inheritance is nominal rather than operational. The relationship-as-first-class half is treated in the companion note A2.17.

These connections are not new commitments. They follow from treating substrate-level preservation as having independent operational content, and they show why the commitment cannot be partially relaxed without the relaxation propagating into multiple other parts of the architecture.

6. Failure modes that violate substrate-level preservation

A substrate can fail substrate-level preservation specifically, even when surrounding components behave correctly. The following failure modes name common ways the violation occurs in practice. The list is not exhaustive; each describes a category of mechanism that can recur in many specific forms.

(a) Last-writer-wins resolution at write time. The substrate accepts a new write that contradicts existing content by replacing the existing content with the new write. The contradiction is silently resolved by the write-time mechanism; the substrate carries only the most recent write.

(b) Deduplication routines. Automated processes scan the substrate for "duplicate" or "redundant" content and consolidate it. Contradictions that were architecturally distinct content are silently collapsed by a dedup pass operating without architectural authority to make collapse decisions.

(c) Merge-on-write. The substrate's write mechanism attempts to merge new writes with related existing content. Contradictions between the new write and existing content are silently merged into a synthesized result that reflects neither original piece accurately.

(d) LLM-mediated silent resolution. An LLM mediator reads contradicting substrate content and proceeds with one interpretation as if the other did not exist, then writes its output back to substrate without recording the contradiction it encountered. The substrate state may technically still contain both contradicting pieces, but the LLM's output is now substrate content derived from a silent resolution that bypassed the contradicting one without architectural acknowledgement; the downstream substrate is incoherent with the contradiction it ostensibly preserves.

(e) Consistency reconciliation. Automated processes reconcile contradicting substrate content to achieve consistency — distributed-system reconciliation, schema validation that rejects contradicting writes at commit time, replication coordinators that converge replicas. Contradictions are eliminated by infrastructure rather than preserved as substrate content.

(f) Hidden by interface filtering with no path back to underlying state. The substrate contains contradictions but standard interfaces filter them out of default views, and no path exists for humans exercising the inspect right to reach the underlying state. The commitment may be technically satisfied at the storage layer but is operationally violated, because no human can exercise authority over contradictions they cannot see. (Where interfaces filter for default-view convenience but a path back to the underlying state remains accessible, the commitment is satisfied; the failure mode is the absence of the path back.)

(g) Garbage collection of "old" contradicting content. Automated processes treat contradicting content as expired or stale and remove it under retention policy. The contradiction is silently resolved by retention rather than by human authority or rule.

A substrate that exhibits any of (a)–(g) does not satisfy substrate-level conflict preservation in the architectural sense, even when the failure happens through reasonable-seeming mechanisms aligned with practices from adjacent architectures.

7. Operational test

A substrate satisfies substrate-level conflict preservation if and only if all of the following are true at all times during the substrate's existence:

1. When two pieces of substrate content contradict each other, both are present in the substrate as addressable substrate content with full provenance.

2. Contradictions persist until either a human with governance authority directly resolves them or an orchestration rule that humans authored handles them at cell execution time. No other resolution path exists architecturally.
3. LLM operations reading substrate content do not modify the substrate as a side effect of reading; the contradicting content remains as it was after LLM reads.
4. Automated processes operating over the substrate — deduplication, merge workers, consistency reconcilers, schema validators, garbage collectors, compaction routines — do not remove or transform contradicting content outside the architectural authority structure.
5. Runtime middleware between humans and the substrate does not transform substrate content as it is read or written in ways that hide or eliminate contradictions.
6. Humans exercising the inspect right can reach contradictions in the underlying substrate state, regardless of whether interfaces filter contradictions in their default views.

A substrate that fails any of (1)–(6) does not satisfy substrate-level conflict preservation in the architectural sense, even if the failure happens through reasonable-seeming mechanisms. Such a substrate may be useful in some other architectural setting, but it is not CKS-coherent on the preservation axis, and downstream work that relies on its preservation guarantees should be scoped accordingly.

8. Conclusion

Implementations under pressure to provide "clean" data drift toward silent contradiction resolution because clean data is technically simpler and aligned with widely adopted database, version-control, and distributed-systems patterns. Substrate-level preservation requires architectural discipline that the natural implementation pressures resist. Drift toward silent resolution produces systems where the substrate's source-of-truth property fails specifically for contradictions: the substrate is authoritative for what was decided, but not for what was contradicted. The failure shows up downstream as decisions made without the full picture of what was disagreed about, retraceability gaps where decisions reference contradictions whose original content is gone, and governance failures where humans cannot exercise authority over contradictions they cannot see.

Naming substrate-level preservation as a standalone architectural commitment gives downstream implementers a precise specification of what their substrate must satisfy independently of how cells behave at execution time. The companion notes A2.14 (cell-level resolution under orchestration rules), A2.15 (the two-level coupling), A2.16 (provenance requirements for first-class contradictions), and A2.17 (the OIDA inheritance) formalize the rest of the joint commitment; together the five notes constitute the operational decomposition of the conflict-as-first-class commitment.

Subsequent work that implements, extends, composes with, or argues against the CKS preservation commitment should use "substrate-level conflict preservation" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Substrate-Level Conflict Preservation as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.